

- ĐỀ CƯƠNG ÔN THI: PHÁT TRIỂN HỆ THỐNG TÍCH HỢP
 - CÂU 1: GIAO TIẾP MẠNG VÀ SOCKET
 - a. Trình bày khái niệm socket trong giao tiếp mạng. Vai trò của socket trong việc thiết lập và quản lý kết nối mạng giữa các ứng dụng client/server. (2,0đ)
 - b. Viết một đoạn mã Java mô tả việc giao tiếp client/server sử dụng TCP socket. Client gửi một tin nhắn dạng chuỗi đến server, và server sẽ phản hồi lại độ dài của tin nhắn đã nhận được. (3,0 điểm)
 - CÂU 2: SO SÁNH TCP VÀ UDP
 - Trình bày sự khác nhau giữa giao thức TCP và UDP trong giao tiếp socket thông qua các tình huống sử dụng điển hình. (3 điểm)
 - CÂU 3: RMI VÀ WEB SERVICE
 - a. Ưu và nhược điểm của RMI (Remote Method Invocation) trong phát triển ứng dụng phân tán? (2 điểm)
 - b. Cách thức hoạt động của Web Service Framework. Ưu điểm của dịch vụ web? (2 điểm)
 - CÂU 4: TRUYỀN THÔNG TCP SOCKET VÀ ĐA LUỒNG
 - a. Trình bày ngắn gọn các giai đoạn chính trong quá trình truyền thông TCP Socket. (6 điểm)
 - b. Xây dựng ứng dụng Chat đa luồng sử dụng TCP Socket
 - BÀI TẬP THỰC HÀNH CÁC DẠNG
 - Dạng 1: Máy Tính Bằng TCP Socket (Bài 8)
 - Dạng 2: Menu Dịch Vụ Đếm Từ (UDP)
 - Dạng 3: TCP Tính Tổng Từng Dòng
 - Dạng 4: Date/Time TCP
 - KIẾN THỨC BỔ SUNG
 - 1. Trình bày khái niệm về Lập trình tích hợp, Hệ thống tích hợp
 - 2. Đặc điểm của hệ thống Publish-Subscribe (Pub/Sub)
 - BÀI TẬP RMI MẪU: ĐẾM ĐỘ DÀI CHUỖI
 - KIẾN THỨC BỔ SUNG: JDBC (JAVA DATABASE CONNECTIVITY)
 - 1. Khái niệm và Vai trò của JDBC
 - 2. Các thành phần chính trong kiến trúc JDBC
 - 3. Các bước cơ bản để kết nối và thao tác với Database bằng JDBC
 - 4. Mã mẫu Đầy Đủ: CRUD (Create, Read, Update, Delete)

ĐỀ CƯƠNG ÔN THI: PHÁT TRIỂN HỆ THỐNG TÍCH HỢP

CÂU 1: GIAO TIẾP MẠNG VÀ SOCKET

a. Trình bày khái niệm socket trong giao tiếp mạng. Vai trò của socket trong việc thiết lập và quản lý kết nối mạng giữa các ứng dụng client/server. (2,0đ)

Khái niệm Socket trong Giao Tiếp Mạng: Socket, hay còn gọi là ổ cắm mạng, là một điểm cuối (endpoint) trong kênh giao tiếp hai chiều giữa hai chương trình chạy trên mạng. Nó cung cấp một giao diện lập trình ứng dụng (API) cho phép các ứng dụng gửi và nhận dữ liệu qua mạng.

Vai trò của Socket trong Kết Nối Client/Server:

- Thiết lập Kết nối:**
 - Client:** Tạo socket và sử dụng nó để kết nối đến địa chỉ và cổng của server.
 - Server:** Tạo socket, liên kết nó với địa chỉ và cổng cụ thể, sau đó lắng nghe kết nối từ client.
- Truyền Dữ liệu:**
 - Client: Gửi dữ liệu đến server thông qua socket.
 - Server: Nhận dữ liệu từ client thông qua socket.
 - Cả hai bên có thể gửi và nhận dữ liệu theo cả hai hướng.
- Quản lý Kết nối:**
 - Kiểm soát luồng:** Socket hỗ trợ các cơ chế kiểm soát luồng để đảm bảo dữ liệu được truyền một cách hiệu quả và tránh quá tải cho bên nhận.
 - Kiểm tra lỗi:** Socket có thể phát hiện lỗi trong quá trình truyền dữ liệu và thực hiện các hành động khắc phục.
 - Đóng kết nối:** Khi hoàn tất việc truyền dữ liệu, socket được đóng để giải phóng tài nguyên.

Các Loại Socket:

- **Socket Dòng (Stream Socket):** Cung cấp luồng dữ liệu đáng tin cậy, hướng kết nối (TCP).
- **Socket Dữ liệu (Datagram Socket):** Cung cấp giao tiếp không kết nối, không đảm bảo độ tin cậy (UDP).

b. Viết một đoạn mã Java mô tả việc giao tiếp client/server sử dụng TCP socket. Client gửi một tin nhắn dạng chuỗi đến server, và server sẽ phản hồi lại độ dài của tin nhắn đã nhận được. (3,0 điểm)

Cách hoạt động:

- **Server:** Tạo socket TCP, lắng nghe, chấp nhận kết nối, nhận dữ liệu, tính toán độ dài và gửi trả lại client.
- **Client:** Tạo socket TCP, kết nối server, nhập tin nhắn, gửi cho server, và nhận độ dài phản hồi.

Server.java

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        final String HOST = "127.0.0.1"; // Địa chỉ loopback
        final int PORT = 65432;          // Cổng sử dụng
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server đang chạy tại " + HOST + ":" + PORT);

            // Chờ client kết nối
            Socket clientSocket = serverSocket.accept();
            System.out.println("Kết nối từ: " + clientSocket.getInetAddress());

            // Xử lý dữ liệu
            try (BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
                PrintWriter out = new PrintWriter(clientSocket.getOutputStream(),
true)) {

                String message;
                while ((message = in.readLine()) != null) {
                    System.out.println("Nhận: " + message);
                    // Tính độ dài tin nhắn và gửi phản hồi
                    int length = message.length();
```

```

        out.println(length); // Gửi độ dài về client
    }
}
} catch (IOException e) {
    System.out.println("Lỗi server: " + e.getMessage());
}
}
}

```

Client.java

```

import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        final String HOST = "127.0.0.1"; // Địa chỉ server
        final int PORT = 65432; // Cổng của server
        try (Socket socket = new Socket(HOST, PORT);
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader userInput = new BufferedReader(new
InputStreamReader(System.in))) {

            System.out.println("Đã kết nối đến server " + HOST + ":" + PORT);
            // Nhập tin nhắn từ người dùng và gửi đến server
            System.out.print("Nhập tin nhắn: ");
            String message = userInput.readLine();
            out.println(message); // Gửi tin nhắn đến server

            // Nhận phản hồi từ server
            String response = in.readLine();
            System.out.println("Độ dài tin nhắn: " + response);

        } catch (IOException e) {
            System.out.println("Lỗi client: " + e.getMessage());
        }
    }
}

```

CÂU 2: SO SÁNH TCP VÀ UDP

Trình bày sự khác nhau giữa giao thức TCP và UDP trong giao tiếp socket thông qua các

tình huống sử dụng điển hình. (3 điểm)

1. TCP (Transmission Control Protocol): TCP là giao thức đảm bảo việc truyền tải dữ liệu một cách an toàn và có thứ tự thông qua cơ chế handshake, kiểm soát luồng, định thời.

- **Kết nối hướng kết nối (Connection-oriented):** Thiết lập một kết nối đáng tin cậy giữa hai máy tính trước khi truyền, đảm bảo đầy đủ và đúng thứ tự.
- **Kiểm soát luồng (Flow control):** Điều chỉnh tốc độ truyền để tránh quá tải cho máy nhận.
- **Kiểm tra lỗi (Error checking):** Phát hiện và sửa lỗi dữ liệu bị hỏng/mất.
- **Thích hợp cho:**
 - Truyền dữ liệu yêu cầu độ tin cậy cao (tải file, email, HTTP, FTP).
 - Ứng dụng cần đảm bảo dữ liệu truyền đầy đủ, đúng thứ tự.

2. UDP (User Datagram Protocol): UDP tập trung vào việc truyền nhanh, hiệu quả mà không cần đảm bảo độ tin cậy. Không yêu cầu thiết lập kết nối (giảm độ trễ), không kiểm soát luồng.

- **Không kết nối (Connectionless):** Không thiết lập kết nối trước, các gói tin (datagram) gửi độc lập, không đảm bảo thứ tự.
- **Không kiểm soát luồng:** Có thể dẫn đến mất mát dữ liệu nếu máy nhận không kịp xử lý.
- **Kiểm tra lỗi cơ bản:** Chỉ kiểm tra lỗi cơ bản của gói tin.
- **Thích hợp cho:**
 - Truyền dữ liệu thời gian thực (streaming video, game online, VoIP).
 - Giao tiếp broadcast và multicast.

Tình huống sử dụng điển hình:

- **TCP:** Tải xuống file (đảm bảo không lỗi).
- **UDP:** Streaming video (nhanh chóng, chấp nhận mất một số khung hình).
- **UDP:** Trò chơi trực tuyến (ưu tiên tốc độ phản ứng).
- **UDP:** DNS (truy vấn tên miền đơn giản, yêu cầu tốc độ).

Tóm lại:

- **TCP:** Độ tin cậy cao, đảm bảo dữ liệu, kiểm soát luồng (ứng dụng cần độ chính xác).
- **UDP:** Tốc độ cao, không đảm bảo dữ liệu (ứng dụng thời gian thực).

CÂU 3: RMI VÀ WEB SERVICE

a. Ưu và nhược điểm của RMI (Remote Method Invocation) trong phát triển ứng dụng phân tán? (2 điểm)

Khái niệm: RMI là công nghệ trong Java cho phép gọi các phương thức của các đối tượng đặt trên máy chủ từ các máy khác. Gồm **Stub** (client) và **Skeleton** (server).

Ưu điểm:

- **Dễ sử dụng:** Đơn giản hóa phát triển, gọi phương thức từ xa như gọi cục bộ.
- **Mạnh mẽ:** Hỗ trợ truyền đối tượng phức tạp, đối tượng tùy chỉnh.
- **Hỗ trợ đa luồng:** Cho phép các client gọi đồng thời.
- **Tích hợp:** Nằm sẵn trong Java, dễ tích hợp với ứng dụng Java.

Nhược điểm:

- **Giới hạn trong Java:** Chỉ hoạt động giữa các ứng dụng Java, không tương thích ngôn ngữ khác.
- **Vấn đề tường lửa:** RMI sử dụng cổng động nên dễ bị chặn bởi Firewall.
- **Khả năng mở rộng:** Khó mở rộng cho ứng dụng cực lớn.
- **Bảo mật:** Dễ bị tấn công nếu cấu hình sai.

Code Triển Khai Nhanh (Calculator):

```
// 1. Remote Interface
import java.rmi.*;
public interface Calculator extends Remote {
    int add(int x, int y) throws RemoteException;
}

// 2. Server
import java.rmi.registry.*;
import java.rmi.server.*;
public class CalculatorServer implements Calculator {
    public CalculatorServer() {}
    public int add(int x, int y) throws RemoteException { return x + y; }
    public static void main(String[] args) {
        try {
            CalculatorServer server = new CalculatorServer();
```

```

        Calculator stub = (Calculator) UnicastRemoteObject.exportObject(server,
0);

        Registry registry = LocateRegistry.createRegistry(1099);
        registry.bind("Calculator", stub);
        System.out.println("Server ready");
    } catch (Exception e) { e.printStackTrace(); }
}

// 3. Client
import java.rmi.registry.*;
public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Calculator calculator = (Calculator) registry.lookup("Calculator");
            System.out.println("Result: " + calculator.add(3, 5));
        } catch (Exception e) { e.printStackTrace(); }
    }
}

```

b. Cách thức hoạt động của Web Service Framework. Ưu điểm của dịch vụ web? (2 điểm)

Cách thức hoạt động:

1. **Định nghĩa Web Service:** Viết mã thực thi và định nghĩa giao diện bằng WSDL (mô tả phương thức, tham số).
2. **Triển khai:** Khởi chạy trên Server có khả năng xử lý SOAP. Công bố file WSDL.
3. **Khám phá:** Client tìm kiếm qua UDDI, tải file WSDL để hiểu giao diện.
4. **Gọi Web Service:** Client đóng gói yêu cầu thành file XML (SOAP Request) và gửi qua mạng (HTTP).
5. **Xử lý yêu cầu:** Server nhận, gọi mã thực thi xử lý.
6. **Trả về kết quả:** Đóng gói kết quả (SOAP Response) và gửi về cho Client.

Ưu điểm của Web Services:

- **Khả năng tương tác:** Dùng chuẩn chung (XML, SOAP, WSDL) giúp mọi ngôn ngữ đều kết nối được.
- **Tái sử dụng:** Dùng được cho nhiều ứng dụng khác nhau.
- **Khả năng mở rộng và truy cập từ xa** qua kết nối Internet tiêu chuẩn (port 80).
- **Đơn giản hóa tích hợp hệ thống.**

CÂU 4: TRUYỀN THÔNG TCP SOCKET VÀ ĐA LUỒNG

a. Trình bày ngắn gọn các giai đoạn chính trong quá trình truyền thông TCP Socket. (6 điểm)

Giao Tiếp TCP Socket giữa Hai Host - Java:

1. **Chuẩn bị:** Import thư viện `java.net.*` và `java.io.*`.
2. **Tạo Socket:**
 - Tự động sử dụng TCP (hướng kết nối).
 - Code: `Socket socket = new Socket("127.0.0.1", 65432);`
3. **Kết nối (Client):**
 - Tự động thực hiện quá trình bắt tay 3 bước (3-way handshake).
4. **Truyền Dữ liệu:**
 - Gửi: Sử dụng `OutputStream` và `PrintWriter`.
 - Nhận: Sử dụng `InputStream` và `BufferedReader`.
5. **Đóng kết nối:**
 - Gọi hàm `close()`. Quá trình bắt tay 4 bước diễn ra để kết thúc an toàn.

b. Xây dựng ứng dụng Chat đa luồng sử dụng TCP Socket

Giải thích: `ChatServer` dùng `ServerSocket` chờ Client, khi có ai kết nối sẽ tạo một Thread `ClientHandler` mới và thêm vào tập hợp `clientWriters`. Khi nhận tin nhắn, Server duyệt `clientWriters` gửi tin cho mọi người.

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class ChatServer {
    private static final int PORT = 65432;
    private static final String HOST = "127.0.0.1";
    private static Set<PrintWriter> clientWriters = ConcurrentHashMap.newKeySet();
```



```

private static class ClientHandler implements Runnable {
    private Socket clientSocket;
    private BufferedReader in;
    private PrintWriter out;

    public ClientHandler(Socket socket) throws IOException {
        this.clientSocket = socket;
        this.in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        this.out = new PrintWriter(socket.getOutputStream(), true);
    }

    @Override
    public void run() {
        try {
            clientWriters.add(out);
            String message;
            while ((message = in.readLine()) != null) {
                System.out.println("Received message: " + message);
                for (PrintWriter writer : clientWriters) {
                    if (writer != out) {
                        writer.println(message);
                    }
                }
            }
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        } finally {
            try { clientSocket.close(); } catch (IOException e) {}
            clientWriters.remove(out);
        }
    }
}

public static void main(String[] args) {
    try (ServerSocket serverSocket = new ServerSocket(PORT)) {
        System.out.println("Server listening on " + HOST + ":" + PORT);
        while (true) {
            Socket clientSocket = serverSocket.accept();
            new Thread(new ClientHandler(clientSocket)).start();
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

BÀI TẬP THỰC HÀNH CÁC DẠNG

Dạng 1: Máy Tính Bằng TCP Socket (Bài 8)

Yêu cầu: Gửi chuỗi dạng + 100 200 để Server tính kết quả.

Serverbai8.java

```
import java.io.*;
import java.net.*;

public class Serverbai8 {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(2003)) {
            System.out.println("Server đang chạy...");
            while (true) {
                Socket clientSocket = serverSocket.accept();
                BufferedReader nhap = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
                PrintWriter xuat = new PrintWriter(new
OutputStreamWriter(clientSocket.getOutputStream()), true);

                String yeuCau;
                while ((yeuCau = nhap.readLine()) != null) {
                    String[] phanTu = yeuCau.split(" ");
                    if (phanTu.length != 3) {
                        xuat.println("Loi dinh dang"); continue;
                    }

                    String phepToan = phanTu[0];
                    double a = Double.parseDouble(phanTu[1]);
                    double b = Double.parseDouble(phanTu[2]);
                    double ketQua = 0;

                    switch (phepToan) {
                        case "+": ketQua = a + b; break;
                        case "-": ketQua = a - b; break;
                        case "*": ketQua = a * b; break;
                        case "/":
                            if(b != 0) ketQua = a / b;
                            else { xuat.println("Loi chia 0"); continue; }
                            break;
                    }
                    xuat.println("Ket qua: " + ketQua);
                }
                clientSocket.close();
            }
        } catch (Exception e) {}
    }
}
```

Clientbai8.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;
```

```

public class Clientbai8 {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 2003);
            BufferedReader nhap = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter xuất = new PrintWriter(new
OutputStreamWriter(socket.getOutputStream()), true);
            Scanner scanner = new Scanner(System.in)) {

            while (true) {
                System.out.print("Nhap phep tinh (VD: 100+200): ");
                String input = scanner.nextLine().trim();
                if (input.equalsIgnoreCase("exit")) break;

                // Format thành chuẩn OP A B
                String yeuCau = formatString(input);
                xuất.println(yeuCau);
                System.out.println("Phan hoi tu server: " + nhap.readLine());
            }
        } catch (Exception e) {}
    }

    private static String formatString(String in) {
        in = in.replaceAll("\\s+", "");
        if (in.contains("+")) return "+" + in.replace("+", " ");
        if (in.contains("-")) return "-" + in.replace("-", " ");
        if (in.contains("*")) return "*" + in.replace("*", " ");
        if (in.contains("/")) return "/" + in.replace("/", " ");
        return null;
    }
}

```

Dạng 2: Menu Dịch Vụ Đếm Từ (UDP)

Yêu cầu: Server UDP đa luồng đếm số từ / ký tự từng dòng. Client gửi kết thúc bằng dấu chấm ..

UDPServer.java

```

import java.io.*;
import java.net.*;

public class UDPServer {
    public static void main(String[] args) throws Exception {
        DatagramSocket serverSocket = new DatagramSocket(9771);
        System.out.println("Server tao tai cong 9771");

        while (true) {
            byte[] receiveData = new byte[4096];
            DatagramPacket receivePacket = new DatagramPacket(receiveData,

```

```

receiveData.length);
    serverSocket.receive(receivePacket);

    String msg = new String(receivePacket.getData(), 0,
receivePacket.getLength());
    if (msg.endsWith(".")) {
        handleClient(msg, receivePacket.getAddress(),
receivePacket.getPort(), serverSocket);
    }
}

private static void handleClient(String msg, InetAddress ip, int port,
DatagramSocket socket) throws Exception {
    StringBuilder response = new StringBuilder();
    String data = msg.substring(0, msg.length() - 1); // Bỏ dấu chấm
    String[] lines = data.split("\n");

    if (msg.startsWith("1")) {
        for (int i = 1; i < lines.length; i++) {
            int count = lines[i].trim().split("\\s+").length;
            response.append("Dong ").append(i).append(" co
").append(count).append(" tu\n");
        }
    } else if (msg.startsWith("2")) {
        for (int i = 1; i < lines.length; i++) {
            int count = lines[i].replaceAll("\\s", "").length();
            response.append("Dong ").append(i).append(" co
").append(count).append(" ky tu\n");
        }
    }

    byte[] sendData = response.toString().getBytes();
    socket.send(new DatagramPacket(sendData, sendData.length, ip, port));
}
}

```

UDPClient.java

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class UDPClient {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket();
        InetAddress ip = InetAddress.getByName("localhost");
        Scanner sc = new Scanner(System.in);

        System.out.print("Chon dich vu (1-Dem tu, 2-Dem ky tu): ");
        String choice = sc.nextLine();

        System.out.println("Nhap van ban (Ket thuc bang dau .): ");
        StringBuilder input = new StringBuilder(choice + "\n");
    }
}

```

```

String line;
while (!(line = sc.nextLine()).endsWith(".")) {
    input.append(line).append("\n");
}
input.append(line); // Thêm dấu chấm cuối

byte[] sendData = input.toString().getBytes();
socket.send(new DatagramPacket(sendData, sendData.length, ip, 9771));

byte[] recvData = new byte[4096];
DatagramPacket recvPacket = new DatagramPacket(recvData, recvData.length);
socket.receive(recvPacket);

System.out.println("Server phản hồi:\n" + new String(recvPacket.getData(),
0, recvPacket.getLength()));
socket.close();
}
}

```

Dạng 3: TCP Tính Tổng Từng Dòng

TCPServer.java (Xử lý nhiều chuỗi số nguyên)

```

// Logic bên trong ClientHandler Thread
String clientInput;
int lineNumber = 0;
while ((clientInput = inFromClient.readLine()) != null) {
    lineNumber++;
    int totalSum = 0;
    boolean isEnd = clientInput.trim().endsWith(".");

    if(isEnd) clientInput = clientInput.substring(0, clientInput.length() - 1);

    String[] numbers = clientInput.split(" ");
    for (String n : numbers) {
        if (!n.isEmpty()) totalSum += Integer.parseInt(n);
    }

    outToClient.writeBytes("Tong dong " + lineNumber + ": " + totalSum + "\n");
    if (isEnd) break;
}

```

Dạng 4: Date/Time TCP

Server nhận Menu 1, 2, 3 và trả về `SimpleDateFormat`:

```
// Logic DateTime Server
switch (clientChoice) {
    case "1": return new SimpleDateFormat("HH:mm:ss").format(new Date());
    case "2": return new SimpleDateFormat("yyyy-MM-dd").format(new Date());
    case "3": return new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new
Date());
}
```

KIẾN THỨC BỔ SUNG

1. Trình bày khái niệm về Lập trình tích hợp, Hệ thống tích hợp

- **Lập trình tích hợp:** Là quá trình kết hợp và tương tác giữa các thành phần phần mềm hoặc các ứng dụng khác nhau để làm cho chúng hoạt động cùng nhau một cách hiệu quả.
- **Hệ thống tích hợp:** Là mô hình hệ thống được xây dựng để tự động hóa và quản lý quá trình kết nối, trao đổi dữ liệu liên mạch.

2. Đặc điểm của hệ thống Publish-Subscribe (Pub/Sub)

- **Giao tiếp tách rời:** Publisher và Subscriber không biết về nhau, chỉ tương tác qua Topic.
- **Bất đồng bộ:** Giảm độ trễ, tăng hiệu quả.
- **Dựa trên Topic:** Gửi và nhận tin nhắn dựa trên các chủ đề.
- **Một - Nhiều:** 1 Publisher gửi cho nhiều Subscriber cùng lúc.
- **Khả năng mở rộng tốt.**

3 Ví dụ thực tế Pub/Sub:

1. **Hệ thống tin tức (Real-time news):** Đăng ký nhận tin tức thể thao, khi có tin hệ thống tự đẩy về cho ai đăng ký Topic đó.
2. **Theo dõi biến động chứng khoán:** Publisher liên tục cập nhật giá, người chơi chứng khoán chỉ đăng ký nghe ngóng đúng mã cổ phiếu của mình.

3. **Mạng IoT:** Hàng ngàn cảm biến báo nhiệt độ về Server. Các module phân tích tự đăng ký nhận data nếu nhiệt độ thay đổi.
-

BÀI TẬP RMI MẪU: ĐẾM ĐỘ DÀI CHUỖI

1. Interface

```
import java.rmi.*;
public interface MyRemoteInterface extends Remote {
    int stringLength(String input) throws RemoteException;
}
```

2. Object Implementation

```
import java.rmi.server.*;
public class MyRemoteObject extends UnicastRemoteObject implements
MyRemoteInterface {
    public MyRemoteObject() throws RemoteException { super(); }
    public int stringLength(String input) throws RemoteException {
        return (input != null) ? input.length() : 0;
    }
}
```

3. Server

```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
public class Server {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            Naming.rebind("rmi://localhost/StringService", new MyRemoteObject());
            System.out.println("Server is ready!");
        } catch (Exception e) {}
    }
}
```

4. Client

```
import java.rmi.Naming;
import java.util.Scanner;
```

```
public class Client {  
    public static void main(String[] args) {  
        try {  
            MyRemoteInterface obj = (MyRemoteInterface)  
Naming.lookup("rmi://localhost/StringService");  
            Scanner sc = new Scanner(System.in);  
            System.out.print("Nhập chuỗi: ");  
            System.out.println("Độ dài: " + obj.stringLength(sc.nextLine()));  
        } catch (Exception e) {}  
    }  
}
```

KIẾN THỨC BỔ SUNG: JDBC (JAVA DATABASE CONNECTIVITY)

1. Khái niệm và Vai trò của JDBC

JDBC (Java Database Connectivity) là một API của Java dùng để kết nối và thực thi các câu lệnh truy vấn tới cơ sở dữ liệu (Database). Nó đóng vai trò như một cầu nối giúp các ứng dụng viết bằng ngôn ngữ Java có thể tương tác (thêm, sửa, xóa, truy vấn) với bất kỳ hệ quản trị cơ sở dữ liệu quan hệ nào (như MySQL, SQL Server, Oracle, PostgreSQL...).

2. Các thành phần chính trong kiến trúc JDBC

- **DriverManager:** Lớp quản lý danh sách các trình điều khiển cơ sở dữ liệu (Database Drivers). Nhiệm vụ của nó là nhận yêu cầu kết nối từ ứng dụng và tìm Driver phù hợp để xử lý.
- **Driver:** Interface xử lý việc giao tiếp cụ thể với từng loại cơ sở dữ liệu.
- **Connection:** Interface đại diện cho toàn bộ phiên làm việc (session) với cơ sở dữ liệu. Nó cho phép tạo ra các đối tượng thực thi câu lệnh SQL.
- **Statement / PreparedStatement / CallableStatement:** Dùng để gửi câu lệnh SQL tới CSDL. PreparedStatement thường được ưu tiên dùng vì nó có khả năng ngăn chặn tấn công SQL Injection và thực thi nhanh hơn.
- **ResultSet:** Đối tượng chứa danh sách các bản ghi (records) trả về sau khi thực hiện câu lệnh SELECT.

3. Các bước cơ bản để kết nối và thao tác với Database bằng JDBC

1. **Nạp/Đăng ký Driver:** (Từ bản JDBC 4.0 trở đi bước này thường được tự động hóa).
2. **Tạo kết nối (Connection):** Sử dụng `DriverManager.getConnection(URL, User, Password)` để mở kết nối.
3. **Tạo Statement/PreparedStatement:** Dùng đối tượng `Connection` để khởi tạo.
4. **Thực thi truy vấn (Execute Query):**
 - Dùng `executeQuery()` cho lệnh `SELECT` (trả về đối tượng `ResultSet`).
 - Dùng `executeUpdate()` cho lệnh `INSERT`, `UPDATE`, `DELETE` (trả về số dòng bị thay đổi).
5. **Xử lý kết quả:** Duyệt qua vòng lặp của `ResultSet` (nếu là lệnh `SELECT`).
6. **Đóng kết nối (Close):** Đóng `ResultSet`, `Statement`, và `Connection` để tránh rò rỉ bộ nhớ.

4. Mã mẫu Đầy Đủ: CRUD (Create, Read, Update, Delete)

Dưới đây là một `Class` tổng hợp đầy đủ 4 thao tác cơ bản với cơ sở dữ liệu. Nếu đi thi yêu cầu viết đoạn code cập nhật, xóa hay lấy dữ liệu, bạn chỉ cần chọn đúng hàm tương ứng để viết:

```
import java.sql.*;

public class JDBC_CRUD_Thi {

    // 1. Hàm dùng chung: Tạo kết nối CSDL (Luôn phải có)
    public static Connection getConnect() throws Exception {
        String url = "jdbc:mysql://localhost:3306/ten_csdl";
        String user = "root";
        String password = "password123";
        return DriverManager.getConnection(url, user, password);
    }

    // 2. CREATE (Thêm mới - INSERT)
    public static void themSinhVien(String ten, int tuoi, String nganh) {
        String sql = "INSERT INTO SinhVien(ten, tuoi, nganh_hoc) VALUES (?, ?, ?)";
        try (Connection conn = getConnect();
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, ten);
```

```

        ps.setInt(2, tuoi);
        ps.setString(3, ngành);

        int rows = ps.executeUpdate(); // Trả về số dòng thêm thành công
        System.out.println("Thêm thành công: " + rows + " dòng");
    } catch (Exception e) { e.printStackTrace(); }
}

// 3. READ (Đọc/Lấy dữ liệu - SELECT)
public static void xemSinhVien() {
    String sql = "SELECT * FROM SinhVien WHERE ngành_hoc = ?";
    try (Connection conn = getConnect();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, "CNTT");
        ResultSet rs = ps.executeQuery(); // SELECT phải dùng executeQuery

        while (rs.next()) {
            System.out.println(rs.getInt("id") + " - " + rs.getString("ten"));
        }
    } catch (Exception e) { e.printStackTrace(); }
}

// 4. UPDATE (Cập nhật - UPDATE)
public static void capNhatTuoi(int idSinhVien, int tuoiMoi) {
    String sql = "UPDATE SinhVien SET tuoi = ? WHERE id = ?";
    try (Connection conn = getConnect();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, tuoiMoi);
        ps.setInt(2, idSinhVien);

        int rows = ps.executeUpdate(); // Trả về số dòng cập nhật thành công
        System.out.println("Cập nhật thành công: " + rows + " dòng");
    } catch (Exception e) { e.printStackTrace(); }
}

// 5. DELETE (Xóa - DELETE)
public static void xoaSinhVien(int idSinhVien) {
    String sql = "DELETE FROM SinhVien WHERE id = ?";
    try (Connection conn = getConnect();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, idSinhVien);

        int rows = ps.executeUpdate(); // Trả về số dòng xóa thành công
        System.out.println("Xóa thành công: " + rows + " dòng");
    } catch (Exception e) { e.printStackTrace(); }
}
}

```

💡 Mẹo ghi nhớ khi làm bài thi:

- SELECT thì bắt buộc dùng `executeQuery()` và phải duyệt lấy dữ liệu thông qua đối tượng `ResultSet` bằng vòng lặp `while(rs.next())`.

- INSERT, UPDATE, DELETE thì dùng chung một hàm là `executeUpdate()`. Hàm này sẽ trả về 1 số nguyên `int` đại diện cho số dòng đã bị thay đổi trong CSDL.
- Luôn sử dụng `PreparedStatement` thay vì `Statement` bằng cách đặt các dấu `?`, sau đó set tham số để code bảo mật (chống SQL Injection) và chuẩn chỉnh hơn.